

Object-Relational Mappers

The pet database layer, implemented with Peewee

Gregory S. DeLozier, PhD

The Route's Interface

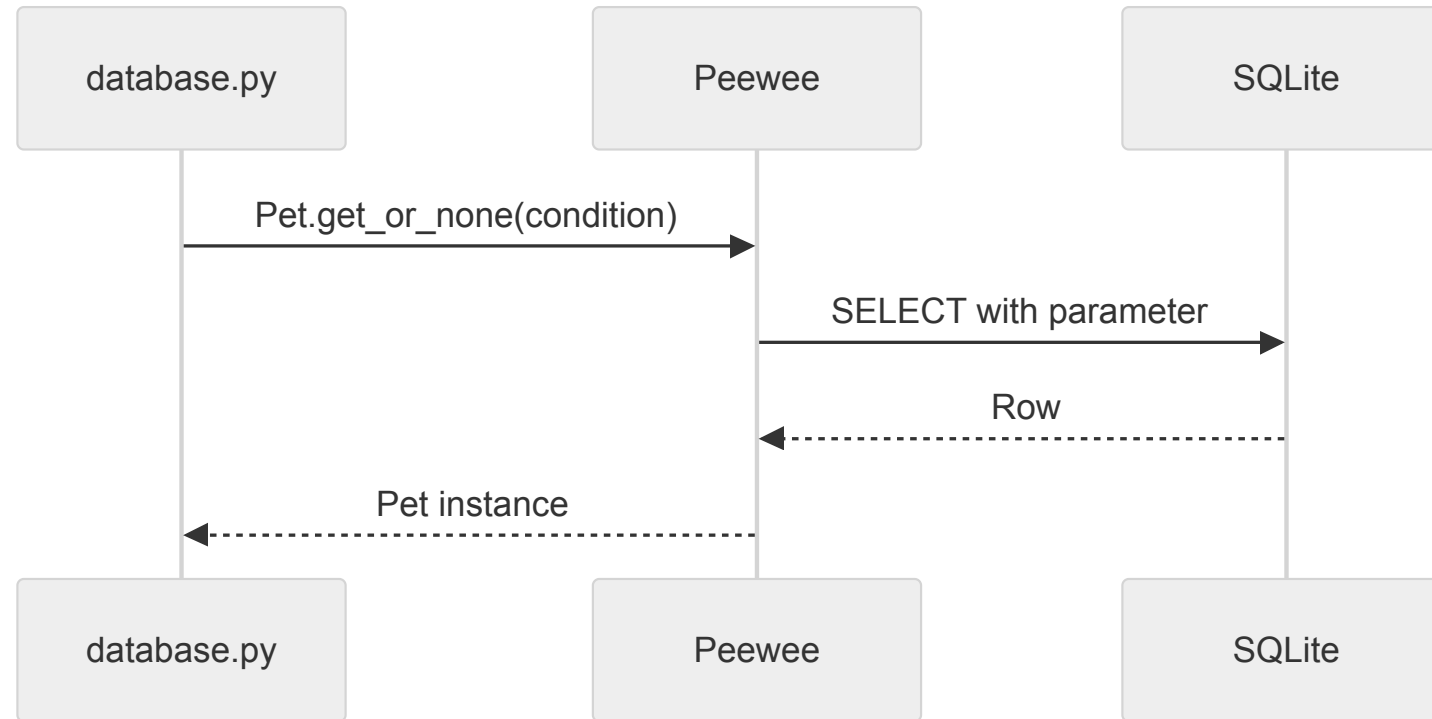
```
data = database.get_pet(id)
```

Input: a pet identifier

Output: a dictionary, or None

The implementation now uses an object-relational mapper (ORM).

A Lookup Through Peewee



Peewee generates Structured Query Language (SQL).

Running the Example

```
python3 -m pip install -r requirements.txt  
flask run
```

`http://127.0.0.1:5000/list`

Setup also runs separately: `python3 setup_database.py`

The Pet Model

```
class Pet(BaseModel):  
    name = TextField(null=False)  
    type = TextField(null=False)  
    age = IntegerField(default=0)  
    food = TextField(null=True)
```

Peewee supplies the integer primary key, **id**.

Class, Field, Instance

Expression	Meaning
Pet	Model class for the table
Pet.name	Field used to build a query
pet.name	Name on one instance
pet.id	Identifier of that pet

Pet describes records. pet holds one record's values.

Connecting the Model

```
db = SQLiteDatabase(None)
```

```
class BaseModel(Model):  
    class Meta:  
        database = db
```

```
db.init("pets.db")  
db.connect(reuse_if_open=True)  
db.create_tables([Pet])
```

Reading the List

```
def get_pets():  
    query = Pet.select().order_by(Pet.name, Pet.id)  
    return [pet_to_dict(pet) for pet in query]
```

Query construction: `select().order_by(...)`

Execution: iteration over query

Reading One Pet

```
pet = Pet.get_or_none(Pet.id == int(id))  
if pet is None:  
    return None  
return pet_to_dict(pet)
```

`Pet.id == int(id)` builds a **SQL condition**.

Returning a Dictionary

```
def pet_to_dict(pet):  
    return {  
        "id": pet.id, "name": pet.name,  
        "type": pet.type, "age": pet.age,  
        "food": pet.food,  
    }
```

The templates keep using `pet['food']`.

Creating a Pet

```
def create_pet(data):  
    pet = Pet.create(**_pet_values(data))  
    return pet.id
```

```
Pet.create(name="Casey", type="dog",  
          age=9, food="kibble")
```

`create()` inserts and returns the **saved instance**.

Updating Casey's Food

```
database.update_pet(10, {  
    "name": "Casey", "type": "dog", "age": "9",  
    "food": "chicken"  
})
```

Record	Before	After
Casey, id=10	food="kibble"	food="chicken"

Executing the Update

```
Pet.update(**_pet_values(data)).where(  
    Pet.id == int(id)  
)
```

Part	Job
update(...)	Replacement values
where(...)	Which pet
execute()	Run the statement

Deleting One Pet

```
Pet.delete().where(Pet.id == int(id)).execute()
```

The **condition** selects the record.

The **execution** performs the deletion.

Object Changes and Saved Values

```
pet.food = "chicken"
```

Action	Object's food	Stored food
Load Casey	kibble	kibble
Assign pet.food	chicken	kibble
Call pet.save()	chicken	chicken

Grouping Writes

```
with database.db.atomic():  
    database.create_pet(casey_data)  
    database.create_pet(heidi_data)
```

Successful block: **commit both.**

Exception escaping the block: **roll back both.**

Inspecting the SQL

```
query = Pet.select().where(Pet.type == "cat")  
statement, parameters = query.sql()  
print(statement)  
print(parameters)
```

Where is the **condition**? Where is the **value**?

Testing Persistence

```
python3 database.py  
python3 -m unittest -v
```

Check

Close, reopen, read

Assign, read, save, read

Raise inside `atomic()`

What it establishes

Values reached the database

When object changes persist

Grouped writes roll back

What Should Cross the Boundary?

`get_pet(id)` could return a dictionary or a Peewee instance.

What changes for the caller?

Keep the return contract explicit when changing the implementation.